

Document number: P0561R0

Date: 2017-02-03

Reply to: Geoff Romer <gromer@google.com>, Andrew Hunter <ahh@google.com>

Audience: Concurrency Study Group

An RAII Interface for Deferred Reclamation

1. [Background](#)
2. [Design Overview](#)
 1. [Read API](#)
 2. [Update API](#)
 3. [Other Design Notes](#)
3. [Acknowledgements](#)

Background

For purposes of this paper, *deferred reclamation* refers to a pattern of concurrent data sharing with two components: *readers* access the data while holding reader locks, which guarantee that the data will remain live while the lock is held. Meanwhile the *updater* updates the data by replacing it with a newly-allocated value. All subsequent readers will see the new value, but the old value is not destroyed until all readers accessing it have released their locks. Readers never block the updater or other readers, and the updater never blocks readers.

This pattern can be implemented in several ways, including reference counting, RCU, and hazard pointers. Several of these have been proposed for standardization (see e.g. [P0233R2](#), [P0279R1](#), and [P0461R0](#)), but the proposals have so far exposed these techniques through fairly low-level APIs.

In this paper, we will propose a high-level RAII API for deferred reclamation, which emphasizes safety and usability rather than fidelity to low-level primitives, but still permits highly efficient implementation. This proposal is based on an API which is used internally at Google, and our experience with it demonstrates the value of providing a high-level API: we provide both a high-level API like the one proposed here, and a more bare-metal RCU API comparable to [P0461](#), and while the high-level API has over 200 users, the low-level API has one.

This proposal is intended to complement, rather than conflict with, proposals for low-level APIs for RCU, hazard pointers, etc, and it can be standardized independently of whether and when we standardize those other proposals.

Underlying API assumptions

Although this proposal is designed to be independent of the low-level API used to implement it, it is necessary to make a few key assumptions. First of all, this design assumes that user code is not required to periodically enter a "quiescent" state where no reader locks are held. We see no way to satisfy such a requirement in a general-purpose library, since it means that any use of the library, no matter how local, imposes constraints on the global structure of the threads that use it (even though the top-level thread code may otherwise be completely unaware of the library). This would be quite onerous to comply with, and likely a source of bugs. Neither [Userspace RCU](#), Google's internal RCU implementation, nor [P0233R2](#)'s hazard pointer API impose this requirement, so omitting it does not appear to be a major implementation burden.

This proposal also does not require user code to register and unregister threads with the library, for more or less the same reasons: it would cause local uses of the library to impose global constraints on the program, creating an unacceptable usability and safety burden. [P0233R2](#) and Google's internal RCU do not impose this requirement, and Userspace RCU provides a library that does not (although at some performance cost). Furthermore, the standard library can satisfy this requirement if need be, without exposing users to it, by performing the necessary registration in `std::thread`.

This proposal requires that when a reader lock is held, a read operation cannot fail to access the underlying data. This appears to preclude an implementation based on hazard pointers, unless augmented with reference counting. Relatedly, this proposal does not bound the amount of data that is ready for reclamation, but isn't yet reclaimed (which hazard pointers can do but RCU cannot). In effect, this API trades off space-efficiency for interface safety and simplicity. We believe this is the right choice for a general-purpose API, reflecting the general programming principle that one should first prefer simple, correct code, and only add complexity when there's a demonstrable performance problem.

Most importantly, this design does *not* assume that the reclamation operation, which schedules cleanup code to be executed asynchronously after all reader locks are released, can allocate memory. In implementations that cannot allocate on reclamation, either reclamation must block, or the caller (in this case the implementation of the high-level API) must supply an object of a library-defined type (such as `rcu_head` in [P0461R0](#) or `hazptr_obj_base` in [P0233R2](#)) that encapsulates whatever

additional storage the library needs to add the callback to its internal data structures, and that object (which we will hereinafter call a "control block") must remain live until the callback is executed. We regard blocking reclamation as an unacceptable safety risk (blocking with a reader lock held can lead to deadlock), so this proposal is designed to accommodate a control block.

Design Overview

The centerpiece of this proposal is the `cell<T>` class template, a wrapper which is either empty or holds a single object of type `T` (the name is intended to suggest a storage cell, but we expect it to be bikeshedded). Rather than providing direct access to the stored object, it allows the user to obtain a *snapshot* of the current state of the object:

```
template <typename T, typename Alloc = allocator<T>>
class cell {
public:
    // Not movable or copyable
    cell(cell&&) = delete;
    cell(const cell&) = delete;
    cell& operator=(cell&&) = delete;
    cell& operator=(const cell&) = delete;

    cell(nullptr_t = nullptr, const Alloc& alloc = Alloc());
    cell(std::unique_ptr<T> ptr, const Alloc& alloc = Alloc());

    update(nullptr_t);
    void update(unique_ptr<T> ptr);

    snapshot_ptr<T> get_snapshot() const;
};

template <typename T>
class snapshot_ptr {
public:
    // Move only
    snapshot_ptr(snapshot_ptr&&);
    snapshot_ptr& operator=(snapshot_ptr&&);
    snapshot_ptr(const snapshot_ptr&) = delete;
    snapshot_ptr& operator=(const snapshot_ptr&) = delete;

    constexpr snapshot_ptr(nullptr_t = nullptr);

    // Converting operations, enabled if U* is convertible to T*
    template <typename U>
    snapshot_ptr(snapshot_ptr<U>&& rhs) noexcept;
    template <typename U>
    snapshot_ptr& operator=(snapshot_ptr<U>&& rhs) noexcept;

    T* get() const noexcept;
    T& operator*() const;
    T* operator->() const noexcept;

    explicit operator bool() const noexcept;
    operator std::shared_ptr<T>() &&;

    void swap(snapshot_ptr& other);
};

template <typename T>
void swap(snapshot_ptr<T>& lhs, snapshot_ptr<T>& rhs);

template <typename T>
bool operator==(const snapshot_ptr<T>& lhs, const snapshot_ptr<T>& rhs);
// Similar overloads for !=, <, >, <=, and >=, and mixed
// comparison with nullptr_t.

template <typename T>
struct hash<snapshot_ptr<T>>;
```

Read API

`get_snapshot()` has a special guarantee that it can be called concurrently with any other operation on the same object (other than construction and destruction, naturally), including non-const operations.

`snapshot_ptr<T>`'s API is closely modeled on `unique_ptr<T>`, and indeed it could be implemented as an alias for `unique_ptr` with a custom deleter, except that we don't want to expose operations such as `release()` or `get_deleter()` that could violate API

invariants or leak implementation details.

A `snapshot_ptr` is either null, or points to a live object of type τ , and it is only null if constructed from `null_ptr`, moved-from, or is the result of invoking `get_snapshot()` on an empty `cell` (in particular, a snapshot cannot spontaneously become null due to the actions of other threads). The guarantee that the object is live means that calling `get_snapshot()` is equivalent to acquiring a reader lock, and destroying the resulting `snapshot_ptr` is equivalent to releasing the reader lock.

All operations on a `snapshot_ptr` are non-blocking.

Open question: should this library require the user to destroy a `snapshot_ptr` in the same thread where it was obtained? The library would be easier to implement with that requirement (because some implementation libraries require lock acquire and release to happen in the same thread), but this would make the library less useful by impeding transfer of data across threads. Note that `unique_lock` implicitly imposes the same requirement, so there is precedent for it.

There are three possible guarantees that this library could make about `snapshot_ptr`s: in increasing order of strictness, it could guarantee that the data object remains live while a `snapshot_ptr` points to it, or it could guarantee that accesses to the data object are race-free, or it could guarantee that the data object is immutable. Note that the liveness guarantee is the bare minimum we require for a deferred reclamation library, and the immutability guarantee is easily implemented by having the library provide only const access to the data object. However, both the liveness and immutability guarantees have substantial drawbacks: providing only a liveness guarantee makes it too easy for users to carelessly instantiate types like `cell<int>` which allow "readers" to cause data races by mutating the data. On the other hand, providing an immutability guarantee disallows use cases like a `cell<atomic<int>>` whose "readers" mutate the data, which we have found to be useful in practice.

Consequently, we argue that this library should be designed to provide at least an informal guarantee of race-freedom, which disallows manifestly hazardous usages like `cell<int>`, but allows safe ones like `cell<atomic<int>>`. This can be achieved by establishing a new type trait `is_race_free<T>`, which is true when concurrent operations on τ cannot cause a data race, and requiring that it be true for `cell`'s template parameter:

```
template <typename T> struct is_race_free : public false_type;

template <typename T>
constexpr bool is_race_free_v = is_race_free<T>::value;

template <typename T>
struct is_race_free<std::atomic<T>> : public true_type;

template <typename T>
struct is_race_free<const T> : public true_type;
```

User code would be permitted to specialize `is_race_free` for user-defined types. The last specialization requires some additional discussion. It expresses the principle that if two operations access the same object through a const access path, they should not cause a data race. This is generally true of C++ standard types (both core and library), but may not be true for all user-defined types, so this specialization may cause some false positives. However, this should be relatively rare, both because it's emphatically a best practice for all types in concurrent code to have this property, and because most types get this property "for free" (to violate it, you generally need to have mutable members or the moral equivalent). Furthermore, if `is_race_free` is erroneously true, nothing actually breaks; we simply fail to prevent the user from instantiating a `cell` type that is easy to misuse. Consequently, we claim that the last specialization is a reasonable compromise that enables many, many valid use cases, and only a few unsafe use cases.

Note, by the way, that the specification of `cell<T>` should probably not explicitly guarantee that accesses to the data are race-free, because this would need to be conditioned on the race-freedom of τ , and that would probably be quite difficult to formally specify. However, the design of `cell` should be geared toward informally enabling users to rely on that guarantee.

The const semantics of `get_snapshot()` merit closer scrutiny. With the API proposed above, users who have only const access to a `cell<T>` can still obtain non-const access to the underlying τ . This is similar to the "shallow const" semantics of pointers, but unlike the "deep const" semantics of other wrapper types such as `optional`. This ensures that the const/non-const distinction matches the reader/updater distinction (readers need only const access, but updaters need non-const access), but at the cost of not capturing the reader/writer distinction (both readers and writers need only const access). In essence, the problem is that this library naturally supports three distinct levels of access (read-only, read-write, and read-write-update), but the const system can only express two. Our intuition is that the writer/updater distinction is more fundamental than the reader/writer distinction, so const should capture the former rather than the latter, but it's a close call.

Open question: When in-place mutation is permitted, should it require non-const access? If so, this could be implemented by providing `get_snapshot()` as a const/non-const overload pair:

```
snapshot_ptr<T> get_snapshot();
snapshot_ptr<const T> get_snapshot() const;
```

Open question: Should `snapshot_ptr` have an aliasing constructor, as `shared_ptr` does? Such a constructor would look like this:

```
template <typename U>
snapshot_ptr(snapshot_ptr<U>&& other, T* ptr);
```

This would enable the user, given a `snapshot_ptr` to an object, to construct a `snapshot_ptr` to one of its members. However, it would mean we could no longer guarantee that a `snapshot_ptr` is either null or points to a live object. Google's implementation does not currently support this, although it has recently been requested.

Update API

The update side is more complex. It consists of two parallel sets of overloads, constructors and `update()`, which respectively initialize the `cell` with a given value, and update the `cell` to store a given value. All `update()` overloads are non-const members, and so by the usual library rules the caller must ensure that they are not called concurrently with any other operations on the `cell` (except `get_snapshot()`, as noted above). `update()` is also always non-blocking, and in particular does not wait for the old data value to be destroyed.

The constructor and `update()` overload taking `nullptr_t` respectively initialize and set the `cell` to the empty state. The fact that a `cell` can be empty is in some ways unfortunate, since it's generally more difficult to reason about types with an empty or null state, and users could always say `cell<optional<T>>` if they explicitly want an empty state. However, given that `snapshot_ptr` must have a null state in order to be movable, eliminating the empty state would not simplify user code much. Furthermore, as will be seen below, every type that can be used to initialize a `cell` naturally has a null value, so forbidding the empty state would actually complicate the API.

The constructor and `update()` overload that accept a `unique_ptr ptr` take ownership of it, and respectively initialize and set the current value of the `cell` to `*ptr`.

It is worth noting that these operations might have to perform allocation (hence the `allocator` parameter) in order to create a control block to subsequently pass to the reclamation operation. Notably, that allocation would be in addition to the allocation that the user presumably performed to create `*ptr`. Now, update operations are inherently costly (because they involve the allocation and construction of an entirely new object), so use cases for the deferred reclamation pattern are generally tolerant of overhead in the update phase, and consequently this extra allocation may not be a substantial concern. However, it's worth considering what it would take to eliminate that overhead.

The obvious answer is that the data object should be allocated together with the control block, in much the same way that `make_shared()` allocates the data object together with the `shared_ptr`'s control block. Naively, we could implement this as an `emplace()` method that takes constructor arguments for `T`, and uses them to allocate and construct a `T` object together with its header. However, this is not a complete solution, because it combines construction of the data object with publication of the data object. In general, users may need to perform some setup on the data object after constructing it, before making it available in the `cell`.

So to solve this in full generality, we would need to provide a way to allocate and construct a `T` object together with the control block, provide mutable access to it, and then store it in a `cell`. This could look something like the following:

```
template <typename T, typename Alloc = std::allocator<T>>
class cell_init {
public:
    // Move only
    cell_init(cell_init&&);
    cell_init& operator=(cell_init&&);
    cell_init(const cell_init&) = delete;
    cell_init& operator=(const cell_init&) = delete;

    template <typename... Args>
    cell_init(Args&& args...);

    template <typename... Args>
    cell_init(allocator_arg_t, const Alloc& alloc, Args&& args...);

    T* get() const;
    T& operator*() const;
    T* operator->() const;
};
```

The constructors forward their arguments to the constructor of `T`, and the smart-pointer-like operations provide mutable access to the data object. We would then simply augment `cell` with a constructor and `update()` overload that take a `cell_init<T, Alloc>` argument.

Open question: Are the performance savings of something like `cell_init` worth the extra complexity it introduces? We have

no implementation/usage experience bearing on this question, because Google's RCU implementation does not take a user-provided control block. Instead, the reclamation operation maintains the cleanup callbacks in an internal data structure akin to a per-CPU `vector`. We believe this to be more efficient (better locality, less cache contention, less space overhead, etc) as well as providing a cleaner API, but it's predicated on the fact that our allocator never fails (it uses memory overcommitment), which means we can allocate inside the reclamation function.

Open question: Should we provide an emplacement-style constructor and updater? As noted, these would not be fully general because users sometimes need to configure the data object before publishing it, but they could provide convenient syntactic sugar, and they would be more efficient than the operations that take `unique_ptr`.

Open question: Should we support taking `unique_ptrs` with non-default deleters? If so, should we make the deleter type a template parameter of `cell`, or type-erase it? Both choices have drawbacks: making the deleter a template parameter prevents us from providing emplacement-style operations, because we have no way of knowing how to allocate the `T` object such that it can be deallocated by an arbitrary user-supplied deleter. Type erasure can impose nontrivial costs, including storage overhead, extra allocations, and an extra pointer indirection on the read path (which is supposed to be extremely fast). On the other hand, implementations which use a control block may already incur many of the same costs; this requires further investigation.

Open question: Relatedly, if we opt to introduce something like `cell_init`, should we provide `unique_ptr` overloads as well? `unique_ptr` is an ubiquitous vocabulary type for handling dynamically-allocated memory, so this would probably be valuable for users, but it would force the implementation to perform type erasure.

It also bears mentioning that this library may need to impose some restrictions on the allocators it supports. In particular, it may need to require that the allocator's `pointer` type is a raw pointer, or can safely be converted to one, since the implementation layer is unlikely to be able to accept "fancy pointers".

One noteworthy property of `update()` is that there is no explicit support for specifying in the `update()` call how the old value is cleaned up, which we are told is required in some RCU use cases in the Linux kernel. It is possible for the user to approximate support for custom cleanup by using a custom destructor or deleter whose behavior is controlled via a side channel, but this is a workaround, and an awkward one at that. I've opted not to include this feature because use cases for it appear to be quite rare (our internal version of this API lacks this feature, and nobody has asked for it), and because it would substantially complicate the API. It would add an extra `update()` parameter which most users don't need, and which would break the symmetry between constructors and `update()` overloads. More fundamentally, it would raise difficult questions about the relationship between the user-supplied cleanup logic and the original deleter: does the cleanup logic run instead of the deleter, or before the deleter? Neither option seems very satisfactory.

Other API notes

We think that `cell`'s destructor should be non-blocking, and should not require all outstanding `snapshot_ptrs` to be destroyed first. This is motivated by the principle that concurrency APIs should strive to avoid coupling between client threads that isn't mediated by the API; concurrency APIs should solve thread coordination problems, not create new ones. That said, destruction of the `cell` must already be coordinated with the threads that actually read it, so also coordinating with the threads that hold `snapshot_ptrs` to it may not be much additional burden (particularly if we decide to disallow passing `snapshot_ptrs` across threads).

Some deferred reclamation libraries are built around the concept of "domains", which can be used to isolate unrelated operations from each other (for example with RCU, long-lived reader locks can delay all reclamation within a domain, but do not affect other domains). This proposal does not include explicit support for domains, so effectively all users of this library would share a single global domain. So far our experience has not shown this to be a problem, but if necessary domain support can easily be added, by adding the domain as a constructor parameter of `cell` (with a default value, so client code can ignore domains if it chooses).

Acknowledgements

Thanks to Paul McKenney and Maged Michael for valuable feedback on a draft of this paper.